

OptChat personal setup – specification

Oct 4, 2026 · @Martin Bergmann

What it is and why

OptChat is one chat with an AI agent that never ends. The agent remembers all of it. It runs on a home server and you use it from a terminal.

What it is

- **The log.** The harness keeps every message, word for word, in a file. It never deletes a message.
- **The tree.** A low-cost model summarizes each message into one line. Then it merges two lines into one, again and again. The older the messages, the more each line covers.
- **The view.** Each turn, the agent sees approximately 128 KB of these lines. The lines cover the full history.
- **Zoom.** When a line is too short, the agent opens it into the two lines below it. It can go down to the original message.
- **Fresh turns.** Each message starts a new agent call. No old context stays in the model.

Why

Normal agent sessions fill up. Then they are compacted or closed, and details are lost. Long sessions also make the model work less well (context rot). Work and decisions spread across many sessions and tools, and the agent cannot find them.

OptChat solves these problems with one idea: the chat history is the memory.

Benefits

Benefit	How you get it
All history in one place	One log holds every message, decision and piece of code
Facts are easy to find	Any message is a few zoom steps away
No context rot	Each turn starts clean with a view of fixed size
No manual compaction	The tree compresses the history in the background

Benefit	How you get it
Low cost	Most of each request comes from the prompt cache
Instructions stay	Your corrections stay in the memory; the latest one applies
Shorter AGENTS.md	Most rules come from the chat, not from a file
You can read it too	You can walk the tree from the root to any message
Always on	It runs on the Mac mini when your laptop is closed
Any model	You can change the model between turns at no cost

Limits: old details become less clear with time, and summaries can lose facts. Sections 6 and 9 give the controls for these risks.

1. Purpose and scope

This specification gives the requirements for one personal OptChat host. The host is one Mac mini. It supports two or more users. Each user has one OptChat instance. Each instance can do development work.

The design follows the OptChat specification by VictorTaelin ([gist](#)), with the changes in section 6.

This setup is for personal use only. Do not put employer data or work repositories on this host.

2. Hardware

Use one Apple silicon Mac mini. Development work sets the size, not the memory system.

Item	Minimum	Recommended	Reason
Mac mini	Apple silicon	Current Apple silicon model	Runs macOS and computer use
RAM	16 GB	32 GB	Each user runs a harness, a CLI, builds and a browser
Internal SSD	256 GB	512 GB	OS, tools, repositories, build output

Item	Minimum	Recommended	Reason
Network	Wi-Fi	Wired Ethernet	Agent runs all day
Power	Mains	Mains + small UPS	A power loss stops all agents
Display	None	HDMI dummy plug	Computer use needs an active screen

Chat data needs approximately 1 GB per user per year. Repositories and build output need much more space.

Set macOS to restart automatically after a power failure. Set macOS to never sleep.

2.1 Network connection

Use a wired connection if possible. Wi-Fi is sufficient to start, because the workload is API calls and SSH.

1. If the mesh router (TP-Link Deco) has a free Ethernet port, connect the Mac mini to it with a Cat 6 cable. Do not put the Mac mini on the router shelf.
2. To use a wall network socket, connect a laptop to it first. Turn off Wi-Fi. If the laptop gets a `192.168.x.x` address and internet access, the socket is connected.
3. If the laptop gets a `169.254.x.x` address, the socket is not connected. Find the other end in the media cabinet or patch panel. Connect it to the router or a switch.
4. If no wired connection is possible, use Wi-Fi or a second Deco unit near the Mac mini.
5. In the Deco app, reserve a fixed IP address for the Mac mini.
6. For access from outside the home, use Tailscale. Do not open ports on the router.

2.2 First setup

The first boot needs a display and a keyboard for approximately 15 minutes. After that, do all work from the MacBook.

1. Connect a TV or monitor (HDMI) and a USB keyboard and mouse.
2. Complete Setup Assistant. Make the administrator account.
3. Do not sign in with your main Apple Account. Do not use Migration Assistant. These copy private data to the agent host.
4. In System Settings > General > Sharing, turn on Screen Sharing and Remote Login.
5. Disconnect the monitor. Connect the HDMI dummy plug.
6. From the MacBook, connect with the Screen Sharing app or `ssh admin@<name>.local`. Copy your SSH key with `ssh-copy-id`.

3. User accounts

Give each person one standard macOS user account. Each OptChat instance runs as its own user.

- 1. Make one administrator account for maintenance only. Do not run an agent as this user.
- 2. Make one standard account for each person.
- 3. Turn on FileVault.
- 4. Make sure that each home folder is readable only by its owner.
- 5. Give each user a separate browser profile and keychain. Do not share passwords between users.
- 6. Turn on Remote Login (SSH). Use key authentication only. Turn off password login.

Each user has these items:

Item	Path or value
Chat data	<code>~/optchat/chat/</code>
Workspaces	<code>~/work/</code>
Harness service	<code>/Library/LaunchDaemons/optchat.<user>.plist</code> (runs as that user)
MCP port	One unique local port per user
Secrets	macOS keychain of that user

4. Software

Install shared tools once, as administrator. Install user tools in each user account.

Software	Scope	Use
macOS, current version	Host	Operating system
Homebrew	Host	Package manager
Node.js LTS	Host	Runs the harness and the Claude Code CLI
git	Host	Code and chat backup
Claude Code CLI	Each user	The master agent

Software	Scope	Use
tmux	Host	Keeps terminal sessions open over SSH
Language toolchains (Python, Rust, etc.)	Each user, as needed	Development work
Chromium-based browser	Each user	Computer use

Install toolchains per user with version managers (for example `fnm`, `uv`, `rustup`). This prevents one user's update from breaking another user's builds.

Do not compile the harness with Perry. Run it on plain Node.js.

4.1 Node.js packages

The harness uses few packages. Most functions use Node.js built-in modules.

Package	Type	Use
<code>@anthropic-ai/sdk</code>	Runtime	Compactor calls with cache control
<code>@modelcontextprotocol/sdk</code>	Runtime	Local MCP server for <code>zoom</code> , <code>date</code> , <code>search</code>
<code>zod</code>	Runtime	Tool input schemas
<code>typescript</code> , <code>tsx</code> , <code>@types/node</code>	Development	Build and run TypeScript
<code>@anthropic-ai/claude-agent-sdk</code>	Optional	Alternative to starting <code>claude -p</code>

Built-in modules: `fs` (append and `fsync`), `net` (lock), `child_process` (CLI and git), `http` (MCP), `readline` (terminal), `crypto` (MCP secret). Install `ripgrep` with Homebrew for fast `search`.

Commit `package-lock.json`. Use exact versions. Install with `npm ci`.

5. Claude accounts and billing

Each user needs one Claude subscription and one API key. Do not share accounts between users.

Role	Model	Access	Reason
Master agent	Claude Opus 5.5 (Sonnet 5.5 for routine turns)	User's subscription, through the official Claude Code CLI	Most tokens are here; subscription is less expensive
Compactor	Claude Sonnet 5.5, medium effort	User's API key	Needs exact cache control; must not stop at a usage limit

1. Log in to the Claude Code CLI with the user's own subscription.
2. Do not extract the subscription token. Do not send it to the API directly. This is not permitted.
3. Make one API key for each user. Keep it in that user's keychain.
4. Set a monthly spend limit on each API key.
5. Make sure that `ANTHROPIC_API_KEY` is not set when the CLI runs. If it is set, the CLI bills the API, not the subscription.

Anthropic changed the subscription rules for programmatic use several times in 2026. Check the current terms before you start ([help article](#)).

6. Harness

Write the harness in TypeScript on Node.js LTS. Give the OptChat specification to Claude Code and tell it to implement the specification exactly. Then add the changes below.

6.1 Required changes to the specification

Change	What to do	Reason
Raw search	Add a tool <code>search(text)</code> . It returns the ids of matching messages from the raw log.	Zoom cannot find a fact that the summaries dropped.
Tool steps	Log each tool call and its result as one message. Update the compactor prompt to match.	Tool noise must not age the user's decisions.
Kind metadata	Store the message kinds that each node covers. Show them on each view line from this data.	The model writes the "user:" tags. Data from outside must not look like user orders.

Change	What to do	Reason
Redaction	Add a command that replaces one message with a marker. Then rebuild only the nodes above it.	Secrets in the log must be removable.
Confirmation	Stop and ask the user before any payment, order or message to another person.	Prevents injected orders from spending money.

Add these changes before the first real use. The tree is never rebuilt, so later changes are difficult.

6.2 How a turn runs

1. The user types a message in the harness terminal.
2. The harness waits until the compactor has summarized all view lines.
3. The harness renders the view and logs the message.
4. The harness starts a new `claude -p` process. It gives the system prompt, the view and the message. It points the CLI to its local MCP server for `zoom`, `date` and `search`.
5. The harness reads the CLI's streamed output. It prints each entry and logs each reply and tool step.
6. The process stops. The compactor summarizes the new messages in the background.

Check the exact CLI flags with `claude --help`. Use streamed JSON output so that the harness can log each step.

6.3 Service

1. Make one LaunchDaemon for each user. Set `UserName` to that user. Set `KeepAlive` to true.
2. Start the harness inside a named tmux session.
3. To use OptChat, connect with SSH as that user and attach to the tmux session.
4. Bind the MCP server to `127.0.0.1` only. Put a random secret in its URL.

6.4 Files and papers

The OptChat specification handles text messages only. It does not save attachments. Add this flow to the harness.

Inputs: a file sent in the terminal or the messenger bridge, or a link to a paper (for example arXiv or DOI).

1. If the input is a link to a PDF, download the PDF. If the download fails, log the link and the error.
2. Save the file in `~/optchat/files/`. Use the name `YYYY-MM-DD-<short-title>.<ext>`. Do not overwrite an existing file.
3. Extract the full text into a `.txt` file with the same name. Use `pdftotext` for PDF files.
4. Log one `user` message with the title, the local path and the source link (if any).
5. Start a turn. Tell the agent to read the file and reply with a summary: main claim, method, key results, and why the user sent it.

The record and the summary are normal log messages. The compactor summarizes them like all other messages. Do not write to the tree directly.

Requirements:

- `search` must also search the `.txt` files in `~/optchat/files/`.
- The agent must read the `.txt` file in parts if it is longer than the 30,000-character tool result cap.
- Include `~/optchat/files/` in the backup (section 10).
- Do not save files that contain secrets or employer data (section 9).

6.5 Link index

The log keeps every link, but summaries drop most links. Add a link index.

1. When the harness logs a user message, extract all URLs from it.
2. Append one line per URL to `~/optchat/links.jsonl`: date, message id, URL and the words around it.
3. Let `search` read this file.

This step uses no model. When you send a link, add a few words about it. These words stay in the summaries.

6.6 Messenger bridge

The harness uses a terminal. To use a phone, add a messenger bridge. Start with Telegram, because it has an official bot API and needs no open ports.

1. Add a local input endpoint to the harness (Unix socket or HTTP on `127.0.0.1`). It accepts messages and streams replies.
2. Write a bridge process. It sends Telegram messages to the endpoint and sends `talk` replies back. Do not send tool calls and results to the phone.
3. Accept messages only from your own Telegram user id. Ignore all other senders.
4. Show confirmations (section 6.1) as Telegram buttons.

5. Save files sent through Telegram with the flow in section 6.4.
6. Use one bot for each user, or map each sender to one harness.

Telegram bot chats are not end-to-end encrypted. Do not send secrets through the bridge.

7. Development work

The agent does development work in the user's `~/work/` folder. It uses the shell and file tools of the Claude Code CLI.

1. Clone each repository into `~/work/<repo>/`.
2. Give each user a separate GitHub account token or SSH key. Use fine-grained tokens with access to the needed repositories only.
3. Do not give the agent permission to push to protected branches. Let it open pull requests.
4. Put build caches and `node_modules` inside `~/work/`. Clean them every month.
5. Do not let the agent run `sudo`. Do not add agent users to the admin group.
6. Keep the OptChat harness code in a separate repository. Do not let the agent change its own running harness.

When the agent learns something about a repository, it must say it in its reply. Tool output is mostly lost in the summaries.

7.1 Daily workflow

- Review agent work through pull requests on GitHub.
- To edit the files yourself, use VS Code Remote-SSH or Zed remote mode from the MacBook.
- Give each subagent its own git worktree. Two agents must not edit the same files.
- Do agent coding through OptChat. Sessions in Claude Code on the MacBook are not in the OptChat log.
- Use OptChat for long tasks, background tasks and tasks that need history. For many small interactive edits, the IDE is faster.

8. Computer use

Only one user at a time can use computer use. A Mac has one active screen session.

1. Select one user as the computer-use user. Log this user in at the screen and keep the session open.

2. FileVault turns off automatic login. After an unplanned restart, log in once through Screen Sharing. For planned restarts, use `sudo fdesetup authrestart`.
3. Connect an HDMI dummy plug if no display is connected.
4. Give other users shell, file and MCP tools only. If they need a browser, give them a headless browser.
5. Log in to shopping and travel sites only in the computer-use user's browser profile.
6. Do not save card numbers in the browser. Enter payment details manually, or use a card with a low limit.
7. Make sure that the harness asks for confirmation before each purchase (section 6.1).

CAUTION: Computer use reads untrusted web pages. A page can contain text that looks like an order. Never let it pay or send without your confirmation.

8.1 Suitable tasks

Use	Examples
Shopping	Fill a grocery basket, reorder food, compare prices
Travel	Search flights and trains, fill in passenger data, check in
Administration	Download invoices, fill in web forms, cancel subscriptions
Research behind login	Get order history or dashboard data into files
Development	Reproduce a UI bug, check a deploy, test a signup flow

Computer use is slow and uses many tokens. Two-factor login and CAPTCHAs stop it; MitID needs your approval on the phone.

Do not use computer use for Digital Post, e-Boks or bank sites. All data that the agent reads goes into the permanent log.

First tasks for the pilot: price comparisons, invoice downloads, and a grocery basket without checkout.

9. Security rules

The log is permanent and the agent acts on it. Keep dangerous data and access out of it.

Rule	Detail
No main email	Do not connect your main inbox. If you need email, use a separate account, read-only, with confirmation before send.
No password reset path	Do not give the agent access to an inbox that can reset your other accounts.
No secrets in chat	Do not paste passwords, API keys or card numbers into the chat. Use the redaction command if this happens.
Tool output cap	Keep the 30,000-character cap on tool results. Do not log files that contain secrets, such as <code>.env</code> .
No work data	Do not put employer data, partner data or work repositories on this host.
Separate users	Do not let one user's agent read another user's home folder.
Network	Do not open the MCP port or SSH to the internet without a VPN, for example Tailscale.
Updates	Install macOS and CLI security updates every month.

WARNING: Tags such as "user:" in the view are written by a model. Do not trust them for payments or for messages to other persons. Always confirm these actions yourself.

10. Backup and restore

The chat log cannot be made again. The tree can be made again, but only with many model calls. Back up both.

1. Make the harness commit `~/optchat/chat/` and `~/optchat/files/` to git after each turn.
2. Push each user's chat repository to a private remote every day. Use one remote for each user.
3. Encrypt the remote copy, or use a host you trust with personal data.
4. Turn on Time Machine to an external disk for the full host.
5. Every month, restore one user's chat to a test folder. Start a harness on it and make sure that the view loads.

To move to a new host, copy `~/optchat/chat/` and start the harness. The harness rebuilds the view from the log. It does not rebuild the tree.

A private GitHub repository is a valid remote. Each daily log file is much smaller than the GitHub file limit. A private repository is not end-to-end encrypted. Encrypt the chat data before you push it (for example with `git-crypt` or `age`), or use an encrypted `restic` backup. Keep the key offline. Without the key, the backup cannot be read.

11. Pilot and acceptance checks

Run a pilot of 2 weeks with one user and low-risk work. Do not connect payments, email or computer use during the pilot.

- ☐ Two harnesses run at the same time as different users, after a restart.
- ☐ One user's agent cannot read the other user's files.
- ☐ The view stays at approximately 128 KB as the log grows.
- ☐ A turn starts within 30 seconds after the previous turn ends.
- ☐ The usage fields show that each step reads the previous step from cache.
- ☐ The master runs on the subscription. The compactor runs on the API key.
- ☐ Recall test: write 20 facts during week 1. In week 2, ask for each fact. Record how many the agent finds and how many zoom hops it needs.
- ☐ `search` finds a fact that zoom did not find.
- ☐ The redaction command removes a test secret from the log and from all nodes above it.
- ☐ A restore from backup loads the same view.
- ☐ API cost for the compactor is below the spend limit for 2 weeks.

After all checks pass, add a second user. Add computer use last, with purchase confirmation turned on.

Appendix A. Alternatives considered

Option	What it is	Difference from this setup
Grok Bot (xAI)	Hosted always-on agents with a shared cloud computer, desktop and iOS apps	No build and better computer use. Logins and data are on the provider's machine. Memory is not a full log.

Option	What it is	Difference from this setup
OpenClaw	Self-hosted agent with messenger channels and Markdown memory notes	Ready messengers and editable memory. Sessions are still compacted; unsaved details are lost.
Grok chatbot	Cloud chat assistant with memory of selected facts	No access to your machine. Memory keeps only the gist.
Claude Code alone	Coding agent with sessions	No memory between sessions. Context rot in long sessions.

Grok Bot and OptChat can be used together: Grok Bot for logged-in web tasks, OptChat for memory and development. This keeps payment logins off the host that holds the full history.